Modelo documental — Sistema de Gestión Documental

MongoDB 7 (replica set de un nodo, rs0) + GridFS · Octubre 2026

Propósito

Este documento describe cómo se modela la información en MongoDB: colecciones, documentos embebidos, referencias entre colecciones e índices.
Cada página es un diagrama o tabla independiente. El diseño parte de los **patrones de acceso** de la aplicación (archivos y carpetas, permisos, papelera, usuarios, actividad y reportes) y no al revés.
El esquema vivo con los índices reales se encuentra en `docs/erd.md`.

Convenciones del modelo

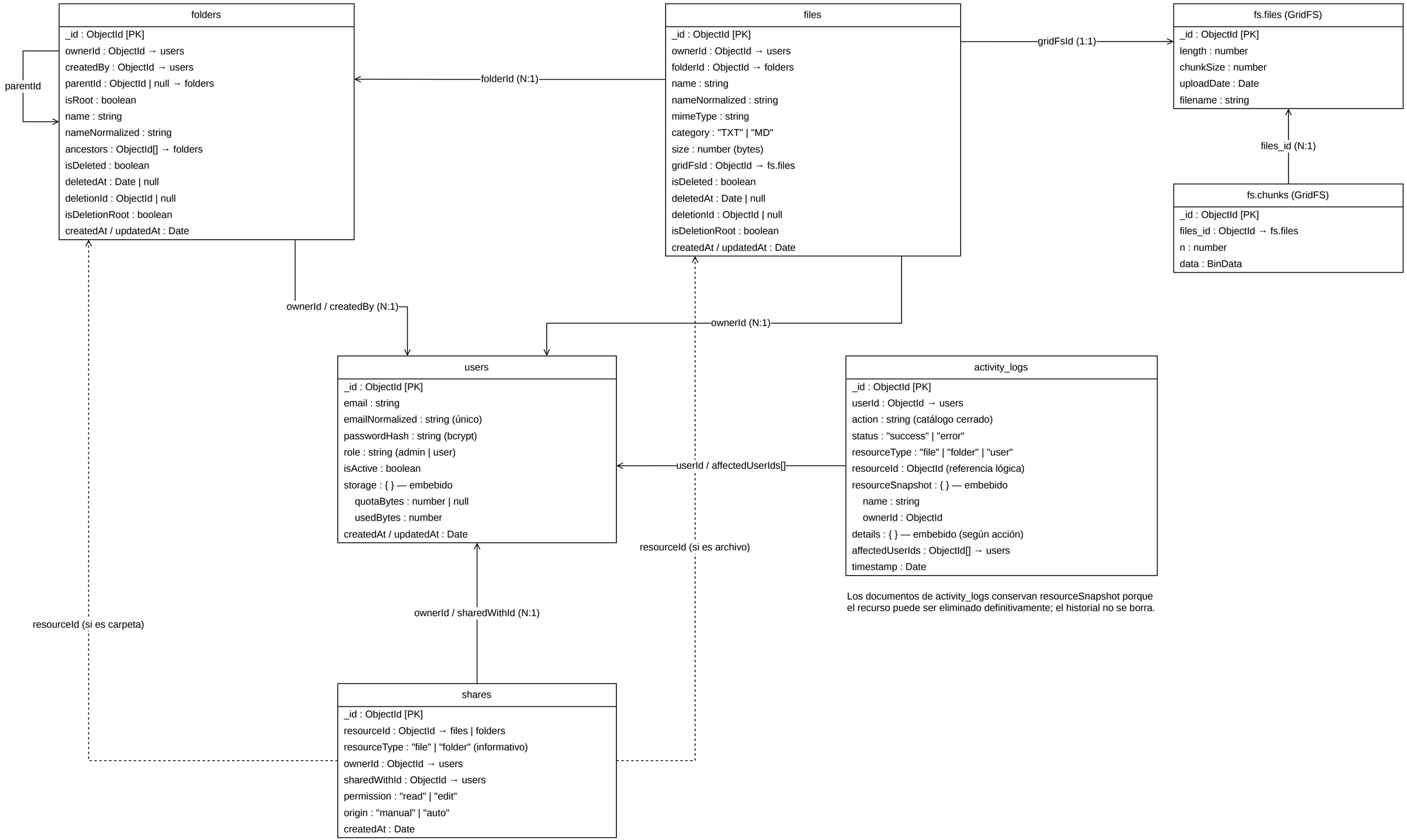
• Las consultas normales de archivos y carpetas llevan `isDeleted: false` (borrado lógico).
• El permiso efectivo se calcula en el servidor con un chequeo centralizado (propietario > share directo > heredado por `ancestors`); el id del usuario sale del `sub` del JWT.
• `name` guarda el nombre mostrado y `nameNormalized` se usa para búsquedas y unicidad (sin mayúsculas ni diacríticos).
• `files.category` solo admite `TXT` o `MD`; el contenido es UTF-8 y se guarda en GridFS.
• La cuota por usuario es configurable y puede ser ilimitada (`quotaBytes: null`).
• Los identificadores se comparan como ObjectId; los emails no se usan como clave porque son mutables.
• MongoDB no impone integridad referencial: las referencias se validan en los servicios y dentro de transacciones.

Cómo leer las páginas

1. Colecciones y relaciones — diagrama entidad-relación con todos los campos.
2. Embebidos vs referencias — qué se embebe, qué se referencia y por qué.
3. Referencias — tabla de cada campo que apunta a otra colección.
4. Índices — todos los índices, sus opciones y la consulta que atienden.
5. Jerarquía y herencia — `parentId`, `ancestors` y permisos heredados.
6. activity_logs — estructura, catálogo de acciones y `affectedUserIds`.
7. GridFS — almacenamiento del contenido y consistencia con la metadata.
8. Ejemplos — documentos de muestra de cada colección.
9. MongoDB en el proyecto — funcionalidades usadas y reglas de los reportes.

Colección	Propósito	Embebidos	Referencias a
`users`	Cuentas, rol, estado activo y cuota de almacenamiento	`storage` {quotaBytes, usedBytes}	—
`folders`	Árbol de carpetas por usuario (una raíz por usuario)	—	users, folders
`files`	Metadatos de archivos `.txt` / `.md`	—	users, folders, fs.files
`shares`	Un documento por recurso y usuario con permiso directo	—	users, files/folders
`activity_logs`	Auditoría de operaciones (éxito y error)	`resourceSnapshot`, `details`	users, recursos (lógica)
`fs.files` / `fs.chunks`	Contenido de los archivos (GridFS)	—	fs.chunks → fs.files

Colecciones y relaciones



Documentos embebidos y referencias

EMBEBIDOS

users.storage

{ quotaBytes: number | null, usedBytes: number }
• Grupo pequeño de tamaño acotado que se lee siempre junto al usuario.
• `quotaBytes: null` significa cuota ilimitada; `usedBytes` se contabiliza igualmente.
• La reserva de cuota es **atómica**: un solo `updateOne` condicionado sobre un único documento. Se hace al terminar la transmisión, para no consumir espacio si la subida falla.
• Se usa al subir y editar archivos, al crear usuarios, al ajustar la cuota y al eliminar definitivamente.

activity_logs.resourceSnapshot

{ name: string, ownerId: ObjectId }
• Fotografía histórica del recurso en el momento de la acción; **no** se sincroniza después.
• Permite mostrar el nombre y resolver el propietario aunque el recurso se elimine definitivamente. Los reportes históricos usan esta copia.

activity_logs.details

Estructura variable según `action` (ver página 6).
• Datos específicos del evento (nombres anterior/nuevo, `gridFsId`, `size`, `deletionId`, errores).
• No es una entidad independiente: no se consulta por separado.
• Guarda `gridFsId` para que el script de limpieza pueda reintentar borrados fallidos.

REFERENCIAS

ownerId / createdBy → users

• Un usuario posee o crea muchos archivos y carpetas: la relación 1:N puede crecer sin límite, por eso no se embebe en `users`.
• `files.ownerId` es quien cargó el archivo y nunca cambia. `folders.ownerId` es siempre el dueño de la carpeta padre.

parentId / folderId / ancestors

• Recursos y jerarquía son documentos independientes; `parentId` es la relación principal.
• `ancestors` es una **ruta materializada** (solo ids) para subárboles, breadcrumb y herencia de permisos con una sola consulta.
• Los archivos no tienen `ancestors`, solo `folderId`: mover una carpeta no obliga a actualizar sus archivos.

shares (colección separada)

• La relación usuarios–recursos es muchos-a-muchos y necesita consultas, actualizaciones y reportes independientes.
• Solo existen **shares directos**. Los permisos heredados se calculan en el servidor y no se almacenan.
• **shares** es la fuente de verdad del acceso; los destinatarios se identifican por `sharedWithId`, no por email.

gridFsId → fs.files

• El contenido físico va en GridFS; `files` solo guarda metadatos y la referencia.
• GridFS no participa en transacciones: metadata y contenido se manejan en fases separadas.

Tabla de referencias entre colecciones

Origen	Campo	Destino	Cardinalidad	Tipo	Notas
`folders`	`ownerId`	`users`	N:1	Referencia	Dueño del árbol; siempre igual al `ownerId` de la carpeta padre.
`folders`	`createdBy`	`users`	N:1	Referencia	Usuario que creó la carpeta (puede ser un editor).
`folders`	`parentId`	`folders`	N:1	Auto-referencia	`null` solo en la carpeta raíz del usuario (`isRoot: true`).
`folders`	`ancestors[]`	`folders`	N:M	Ruta materializada	Solo ids, de la raíz al padre; índice multikey.
`files`	`ownerId`	`users`	N:1	Referencia	Quien cargó el archivo; nunca cambia.
`files`	`folderId`	`folders`	N:1	Referencia	Carpeta contenedora; la ruta se obtiene desde la carpeta.
`files`	`gridFsId`	`fs.files`	1:1	Referencia	Se reemplaza al editar el contenido (objeto GridFS nuevo).
`shares`	`resourceId`	`files` / `folders`	N:1	Polimórfica	Los ObjectId no se repiten entre colecciones; `resourceType` es solo informativo.
`shares`	`ownerId`	`users`	N:1	Referencia	Propietario del recurso compartido.
`shares`	`sharedWithId`	`users`	N:1	Referencia	Destinatario del permiso; debe estar activo.
`activity_logs`	`userId`	`users`	N:1	Referencia	Actor de la acción.
`activity_logs`	`resourceId`	`files` / `folders` / `users`	N:1	Referencia lógica	Puede quedar huérfana tras el borrado definitivo; por eso existe `resourceSnapshot`.
`activity_logs`	`affectedUserIds[]`	`users`	N:M	Referencia	Otros usuarios cuyos recursos o accesos se tocaron; alimenta la actividad que afecta a cada usuario.
`activity_logs.detail`	`gridFsId`	`fs.files`	N:1	Referencia lógica	Para limpieza posterior si falla el borrado en GridFS.
`fs.chunks`	`files_id`	`fs.files`	N:1	Referencia	Estándar de GridFS: los fragmentos pertenecen a un objeto.

Integridad: al subir, mover, compartir o eliminar, los servicios validan existencia, `isDeleted: false` y permisos antes de escribir; las operaciones multi-documento usan transacciones (requieren replica set). `ownerId` de un archivo nunca cambia, por lo que mover entre propietarios está prohibido.

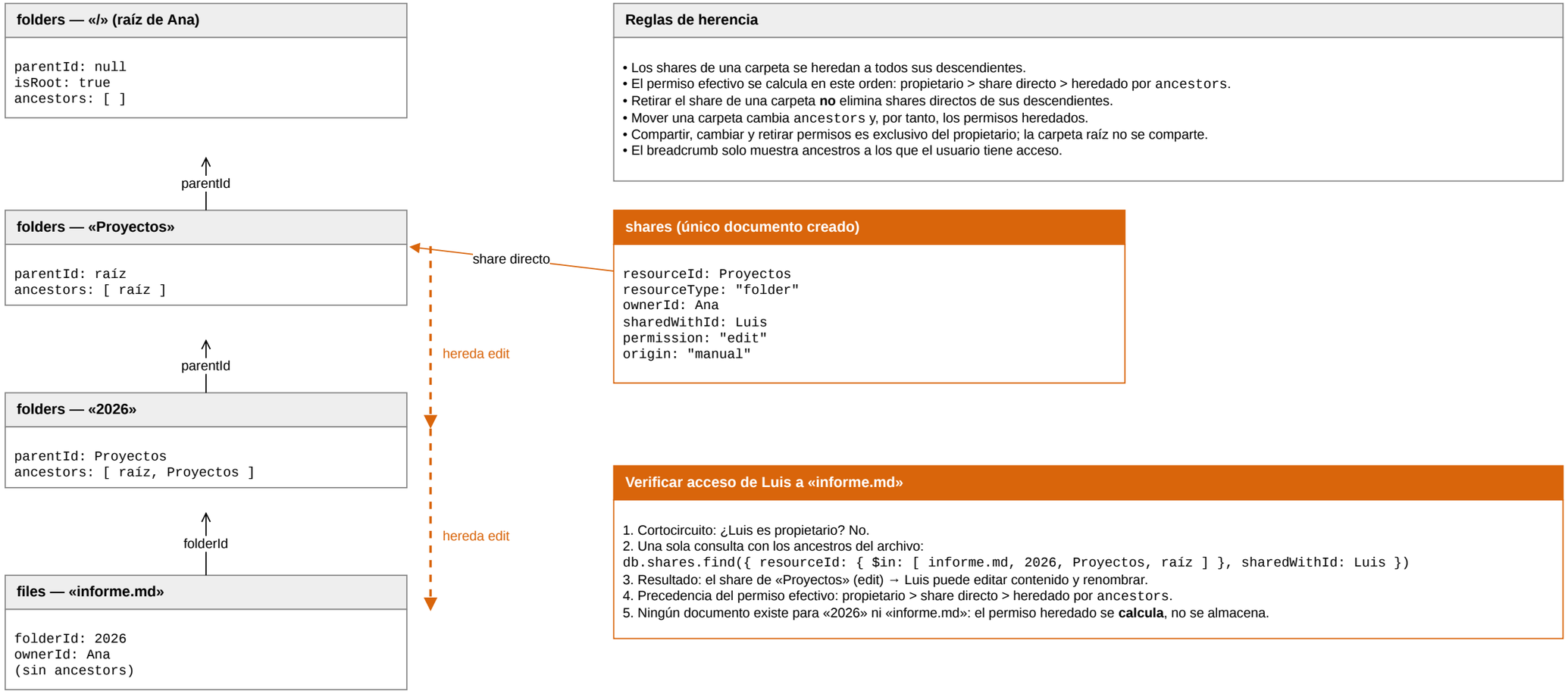
Índices por colección

Cada índice responde a una consulta concreta de la aplicación; la última columna indica cuál. Los compuestos ponen primero los campos de igualdad y después el de orden.

Colección	Índice	Opciones	Consulta que atiende
users	`{ emailNormalized: 1 }`	**unique**	Inicio de sesión; unicidad de correo
users	`{ isActive: 1, role: 1 }`	—	Listado de usuarios
folders	`{ parentId: 1, isDeleted: 1, nameNormalized: 1 }`	—	Listar contenido de una carpeta ordenado por nombre
folders	`{ ancestors: 1, isDeleted: 1 }`	multikey	Mover, papelera, herencia de permisos y breadcrumb
folders	`{ parentId: 1, nameNormalized: 1 }`	**unique** parcial `{isDeleted:false, isRoot:false}`	Nombre único dentro de la carpeta
folders	`{ ownerId: 1 }`	**unique** parcial `{isRoot:true}`	Una sola carpeta raíz por usuario
folders	`{ ownerId: 1, deletedAt: -1 }`	parcial `{isDeleted:true, isDeletionRoot:true}`	Listar papelera
folders	`{ deletionId: 1 }`	parcial `{isDeleted:true}`	Restaurar y eliminar definitivamente
files	`{ folderId: 1, isDeleted: 1, nameNormalized: 1 }`	—	Listar archivos de una carpeta; búsqueda en compartidos
files	`{ folderId: 1, nameNormalized: 1 }`	**unique** parcial `{isDeleted:false}`	Nombre único dentro de la carpeta
files	`{ ownerId: 1, isDeleted: 1, category: 1, createdAt: -1 }`	—	Filtro por tipo y fecha (igualdad, luego orden y rango)
files	`{ ownerId: 1, isDeleted: 1, createdAt: -1 }`	—	Filtro por fecha sin tipo
files	`{ ownerId: 1, isDeleted: 1, nameNormalized: 1 }`	—	Búsqueda en archivos propios y consultas por propietario
files	`{ isDeleted: 1, createdAt: -1 }`	—	Reportes por rango de fechas; conteos del administrador
files	`{ ownerId: 1, deletedAt: -1 }`	parcial `{isDeleted:true, isDeletionRoot:true}`	Listar papelera
files	`{ deletionId: 1 }`	parcial `{isDeleted:true}`	Restaurar y eliminar definitivamente
shares	`{ resourceId: 1, sharedWithId: 1 }`	**unique**	Cálculo de permisos; reportes por recurso
shares	`{ sharedWithId: 1, createdAt: -1 }`	—	Compartidos conmigo; reportes por destinatario
shares	`{ ownerId: 1, origin: 1, createdAt: -1 }`	—	Compartidos por mí; reportes de compartición
activity_logs	`{ userId: 1, timestamp: -1 }`	—	Historial de un usuario
activity_logs	`{ resourceType: 1, resourceId: 1, timestamp: -1 }`	—	Historial de un recurso; reportes de recursos
activity_logs	`{ timestamp: -1 }`	—	Operaciones por período
activity_logs	`{ action: 1, status: 1, timestamp: -1 }`	—	Operaciones por tipo de acción
activity_logs	`{ affectedUserIds: 1, timestamp: -1 }`	multikey	Actividad que afecta al usuario

Búsqueda por nombre: usa regex escapado e insensible a mayúsculas sobre `nameNormalized`; el regex no puede aprovechar el índice, pero sí el prefijo de igualdad (`ownerId`, `parentId` / `folderId`, `isDeleted`).

Jerarquía de carpetas, ancestors y permisos heredados



Reglas de herencia

- Los shares de una carpeta se heredan a todos sus descendientes.
- El permiso efectivo se calcula en este orden: propietario > share directo > heredado por `ancestors`.
- Retirar el share de una carpeta **no** elimina shares directos de sus descendientes.
- Mover una carpeta cambia `ancestors` y, por tanto, los permisos heredados.
- Compartir, cambiar y retirar permisos es exclusivo del propietario; la carpeta raíz no se comparte.
- El breadcrumb solo muestra ancestros a los que el usuario tiene acceso.

Verificar acceso de Luis a «informe.md»

1. Cortocircuito: ¿Luis es propietario? No.
2. Una sola consulta con los ancestros del archivo:
`db.shares.find({ resourceId: { $in: [ informe.md, 2026, Proyectos, raíz ] }, sharedWithId: Luis })`
3. Resultado: el share de «Proyectos» (edit) → Luis puede editar contenido y renombrar.
4. Precedencia del permiso efectivo: propietario > share directo > heredado por `ancestors`.
5. Ningún documento existe para «2026» ni «informe.md»: el permiso heredado se **calcula**, no se almacena.

Colección activity_logs: estructura y catálogo de acciones

Estructura del documento

```
{
_id: ObjectId,
userId: ObjectId, // actor
action: "UPLOAD",
status: "success" | "error",
resourceType: "file" | "folder" | "user",
resourceId: ObjectId,
resourceSnapshot: { name, ownerId },
details: { ... }, // según acción
affectedUserIds: [ObjectId],
timestamp: ISODate
}
```

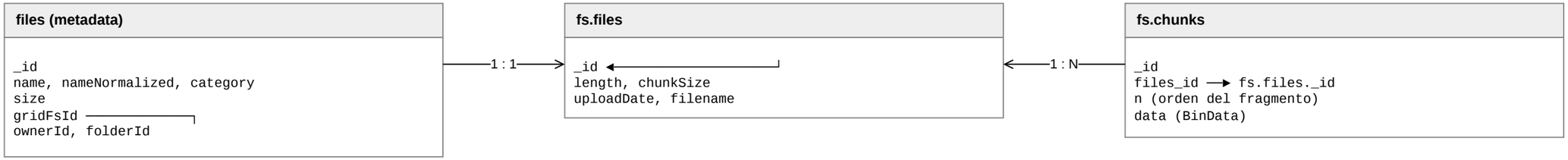
Catálogo cerrado de action

Archivos: `UPLOAD`, `DOWNLOAD`, `EDIT_CONTENT`, `RENAME`, `MOVE`
Carpetas: `FOLDER_CREATE`
Papelera: `TRASH`, `RESTORE`, `DELETE_PERMANENT`
Compartición: `SHARE`, `PERMISSION_CHANGE`, `UNSHARE`
Administración: `USER_CREATE`, `USER_STATUS_CHANGE`, `QUOTA_CHANGE`
Nota: la vista previa **no** registra actividad; no existe la acción `VIEW`.

action	details (ejemplo)	affectedUserIds se llena con
`UPLOAD`	`{ gridFsId, size, folderId }`	Dueño de la carpeta si es ajena
`EDIT_CONTENT`	`{ previousGridFsId, newGridFsId, previousSize, newSize }`	Propietario y usuarios con share directo
`FOLDER_CREATE`	`{ parentId, createdBy }`	Propietario del árbol si actúa otro
`RENAME`	`{ previousName, newName }`	Propietario y usuarios con share directo
`MOVE`	`{ previousFolderId, newFolderId }`	Usuarios con share directo
`SHARE` / `PERMISSION_CHANGE` / `UNSHARE`	`{ sharedWithId, permission, origin }`	El destinatario
`TRASH`	`{ deletionId, isDeletionRoot }`	Propietario y usuarios con share directo
`RESTORE`	`{ deletionId, previousFolderId, destinationFolderId }`	Propietario y usuarios con share directo
`DELETE_PERMANENT`	`{ deletionId, gridFsId, size }`	Propietario y usuarios con share directo
`QUOTA_CHANGE`	`{ previousQuotaBytes, newQuotaBytes }`	El usuario afectado
`status: "error"`	`{ errorCode: "GRIDFS_DELETE_FAILED", gridFsId, message }`	—

Reglas de afectados: `affectedUserIds` se puebla solo con el **propietario** y los **shares directos** de logs nuevos; no se hace backfill masivo sobre logs existentes.
Visibilidad: el historial de un recurso muestra acciones propias y las del propietario del recurso; la actividad que me afecta filtra `affectedUserIds = usuario`, `status: "success"` y excluye acciones propias.
Operaciones masivas: se crea un log por carpeta y por cada recurso afectado.
Conservación: el historial es append-only con `timestamp` explícito y se conserva aunque el recurso se elimine definitivamente.

Almacenamiento del contenido con GridFS



Reserva atómica de cuota

```
db.users.updateOne(
{ _id: ownerId, isActive: true,
$or: [ { "storage.quotaBytes": null },
{ $expr: { $lte: [ { $add: ["$storage.usedBytes", size] }, "$storage.quotaBytes" ] } } ] },
{ $inc: { "storage.usedBytes": size } }, { session })
```

`matchedCount === 0` → cuota excedida o usuario inactivo: abortar la transacción.

Dos fases para mantener consistencia

Carga: (1) stream a GridFS con validación UTF-8 → (2) transacción: reservar cuota + crear `files` + share automático + log → si algo falla, borrar el objeto GridFS.
Edición: (1) objeto GridFS nuevo → (2) transacción: delta de cuota + actualizar `files` + log → (3) borrar el objeto anterior.
Eliminación definitiva: (1) transacción de metadata, shares, cuota y log → (2) borrar objetos GridFS; si falla se registra `GRIDFS_DELETE_FAILED` con el `gridFsId`.

Reglas del contenido

- Solo `.txt` y `.md` (la categoría se deriva de la extensión); MIME permitido: `text/plain`, `text/markdown`, `text/x-markdown`, `application/octet-stream`.
- UTF-8 validado en streaming con `TextDecoder("utf-8", { fatal: true })` y `{ stream: true }`.
- La extensión y el MIME se validan antes de crear el objeto GridFS para no dejar huérfanos.
- Sin límite individual de tamaño; el límite efectivo es la cuota.
- Editar contenido crea un objeto GridFS **nuevo** y reemplaza `gridFsId`; el anterior se borra tras el commit.
- Descargar sirve el stream como adjunto (nombre con fallback ASCII y `filename*` UTF-8); la vista previa lo sirve en línea y no registra actividad.

Limpieza de GridFS huérfano

El script de limpieza (`npm run db:cleanup-gridfs`) lee los logs con `errorCode` de GridFS, reintenta el borrado del `gridFsId` guardado en `details` y trata «archivo no encontrado» como éxito.

Ejemplos de documentos

Valores ilustrativos; los ObjectId se abrevian.

users

```
{
_id: ObjectId("…a1"),
email: "Juan@docsdrive.local",
emailNormalized: "juan@docsdrive.local",
passwordHash: "$2b$10$…",
role: "user",
isActive: true,
storage: { quotaBytes: 104857600, usedBytes: 20480 },
createdAt: ISODate("2026-10-01T15:00:00Z")
}
```

folders (raíz y carpeta normal)

```
{ _id: …b0, ownerId: …a1, createdBy: …a1, parentId: null,
isRoot: true, name: "/", ancestors: [], isDeleted: false }

{ _id: …b1, ownerId: …a1, createdBy: …a2, parentId: …b0,
isRoot: false, name: "Proyectos", nameNormalized: "proyectos",
ancestors: [ …b0 ], isDeleted: false }
```

files

```
{
_id: …c1, ownerId: …a1, folderId: …b1,
name: "informe.md", nameNormalized: "informe.md",
mimeType: "text/markdown", category: "MD",
size: 2048, gridFsId: …d1,
isDeleted: false, deletedAt: null,
createdAt: ISODate("2026-10-05T10:00:00Z")
}
```

shares

```
{
_id: …e1, resourceType: "file", resourceId: …c1,
ownerId: …a1, sharedWithId: …a3,
permission: "read", origin: "manual",
createdAt: ISODate("2026-10-06T09:00:00Z")
}
```

shares (automático por carga en carpeta ajena)

```
{ resourceId: …c2, ownerId: …a2, sharedWithId: …a1,
permission: "edit", origin: "auto" }

// …c2 lo cargó otro usuario en una carpeta de …a1;
// …a1 recibe edit automático.
```

activity_logs (UPLOAD)

```
{
userId: …a2, action: "UPLOAD", status: "success",
resourceType: "file", resourceId: …c2,
resourceSnapshot: { name: "notas.txt", ownerId: …a2 },
details: { gridFsId: …d2, size: 1200, folderId: …b1 },
affectedUserIds: [ …a1 ],
timestamp: ISODate("2026-10-07T14:30:00Z")
}
```

activity_logs (error de GridFS)

```
{
userId: …a1, action: "DELETE_PERMANENT",
status: "error",
resourceType: "file", resourceId: …c1,
details: {
errorCode: "GRIDFS_DELETE_FAILED",
gridFsId: …d1,
message: "No fue posible eliminar…"
}
}
```

MongoDB en el proyecto

Funcionalidades propias de MongoDB que usa el sistema y las reglas que siguen los reportes administrativos.

Funcionalidades de MongoDB usadas

Replica set + transacciones: un replica set de un solo nodo habilita operaciones atómicas entre colecciones: alta de usuario con su carpeta raíz y log, borrado definitivo de metadata, shares y cuota, y movimientos.
GridFS: almacenamiento binario fuera de las colecciones de documentos, con borrado manual; `files.gridFsId` enlaza ambos.
Aggregation pipeline: reportes con `$match`, `$group`, `$sort`, `$project`, `$lookup`, `$unwind`, `$facet` y `$dateTrunc`.
Índices compuestos y parciales: prefijo de igualdad antes del campo de orden, `partialFilterExpression` y unicidad condicional.
Denormalización: `resourceSnapshot` en logs, `ancestors` en carpetas y emails cacheados cuando es necesario.
ObjectId: claves y relaciones entre colecciones; los emails no se usan como clave porque son mutables.

Transacciones y GridFS

• GridFS no participa en transacciones: la metadata se guarda en una transacción y el contenido se crea o elimina fuera de ella; si algo falla se elimina el objeto GridFS para evitar huérfanos.
• La cuota (`users.storage`) se reserva al terminar la transmisión, para no consumir espacio si la subida falla.
• `activity_logs` es append-only: sus registros se conservan aunque el recurso se elimine definitivamente.

Reglas de los reportes (solo administradores)

• Trabajan solo con metadatos; nunca leen el contenido de los archivos.
• Los reportes de operaciones cuentan únicamente registros con `status: "success"`.
• Los reportes de almacenamiento excluyen archivos eliminados.
• Los rankings de compartición incluyen shares manuales y automáticos, salvo el de archivos compartidos con más usuarios, que usa solo manuales.
• Los reportes históricos usan `resourceSnapshot` cuando el recurso ya no existe.
• Las fechas se interpretan en la zona `America/Guatemala` y los rangos son inclusivos.
• Ofrecen listados paginados y selectores de usuarios para filtrar por propietario o destinatario.

Etapa	Uso en los reportes
`$match`	Filtrar por estado, tipo, usuario o rango de fechas
`$group`	Contar y sumar métricas
`$lookup + $unwind`	Unir archivos y usuarios sin consultas N+1
`$facet`	Obtener datos paginados, total y resumen en una sola consulta
`$dateTrunc`	Agrupar operaciones por día, semana o mes en America/Guatemala

Búsqueda y filtros: la búsqueda es unificada para archivos y carpetas con los alcances own, `shared` y `all`. El alcance `shared` incluye shares directos y la herencia por `ancestors`. Los resultados se combinan en Node, se ordenan por nombre y se paginan de forma conjunta; cada uno incluye el permiso efectivo.